A MINI PROJECT REPORT ON

CHANGE DATA CAPTURE (CDC) USING MONGODB AND NODE.JS

Submitted in partial fulfillment of the requirements

Submitted By:

Bhoomi Sharma

2025-2026

1. Abstract

Modern fast-paced applications require data updates to be instantly sent to client interfaces without manual refreshes or heavy database polling. This project, titled **Change Data Capture (CDC)**, proposes a highly optimized event-driven monitoring system. By utilizing MongoDB's Change Streams capability coupled with a backend Node.js streaming architecture, any document changes (inserts, updates, or deletes) are captured in real-time. This ensures that application dashboards hold strictly up-to-date information without incurring any database payload overhead.

2. Problem Statement

Traditional data synchronization relies on "polling" - where the frontend consistently asks the database for updates. This leads to severe database performance bottlenecks, bandwidth wastage, and an inherent delay before data reaches the user. The objective of this project is to implement a **Push-based** CDC system that mitigates server load, captures database changes precisely at the moment of mutation, and accurately batches them to a real-time monitoring dashboard.

3. System Architecture

The system is built sequentially in three highly coupled layers:

- **Database Layer (MongoDB Replica Set):** Stores application data. Contains an integrated pipeline (Change Streams) which continuously emits database operations.
- **Backend Processing Layer (Node.js & Express):** Acts as the listener. It securely buffers events, applies a "debouncing" algorithm to avoid overwhelming the frontend, groups rapid changes into logical batches, and serves them via an API.
- **Frontend Monitoring Layer (HTML, CSS, JS):** Fetches the aggregated batches at a fixed interval and dynamically updates a beautiful glassmorphic dashboard in real-time.

4. Core Implementation: Debouncing & Batching

During traffic spikes (e.g., thousands of database writes per second), immediately emitting every event to the UI would cause application crashes. To solve this, a specialized **DebounceService** was developed. When an event fires, it enters an internal memory buffer. A timer begins counting (the quiet period). If standard capacity is reached, or the timer finishes, all buffered events are merged into a single "Batch" payload. The frontend consumes this perfectly safe payload, successfully decoupling the display layer from direct database speed.

5. Application Outputs

The monitoring dashboard initializes by polling the health API, confirming that the backend buffer is ready and waiting for Mongo operations.

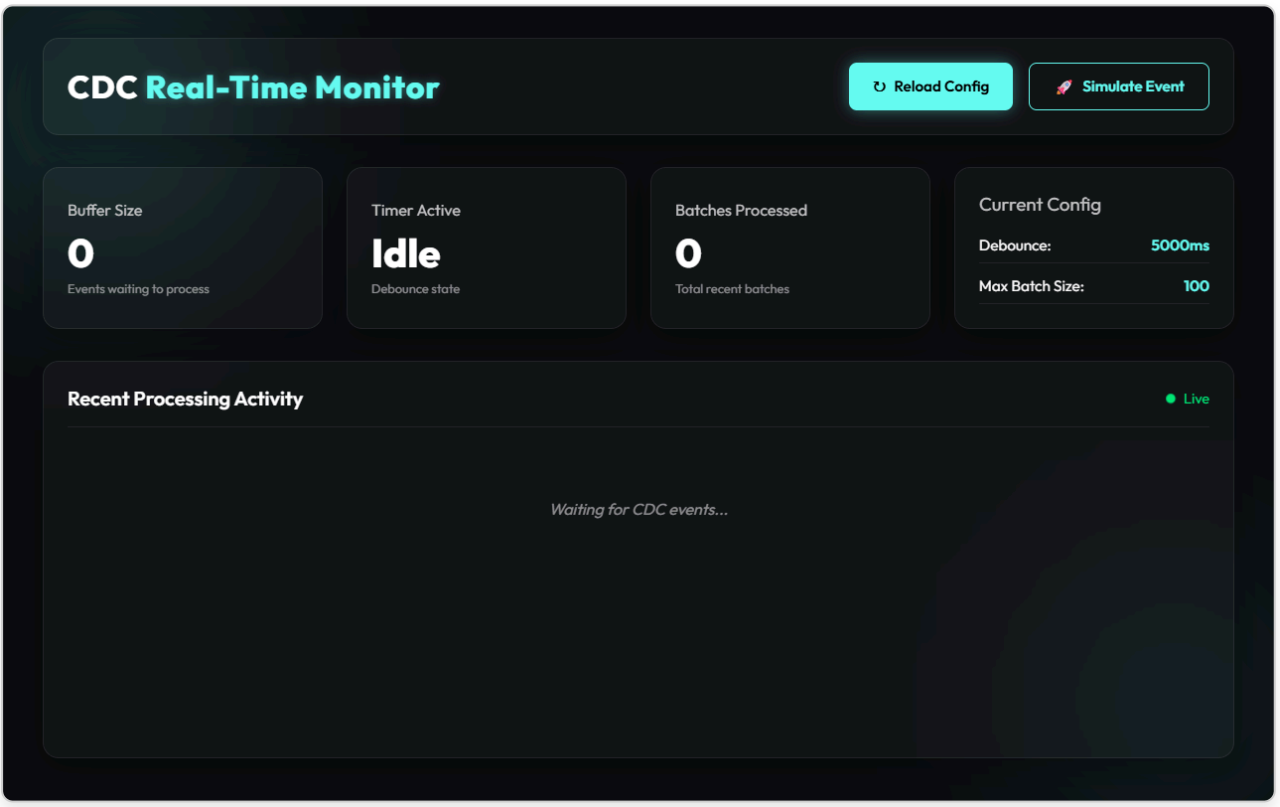

Figure 1: The Initial Real-Time Monitor Dashboard State

To demonstrate the real-time functionality without manual database insertion, a simulated event payload is triggered from the interface. It securely contacts the Node.js backend to emulate a data stream. The buffer collects it, triggers the debounce, and then emits the Batched record down to the frontend—zero page refresh required.

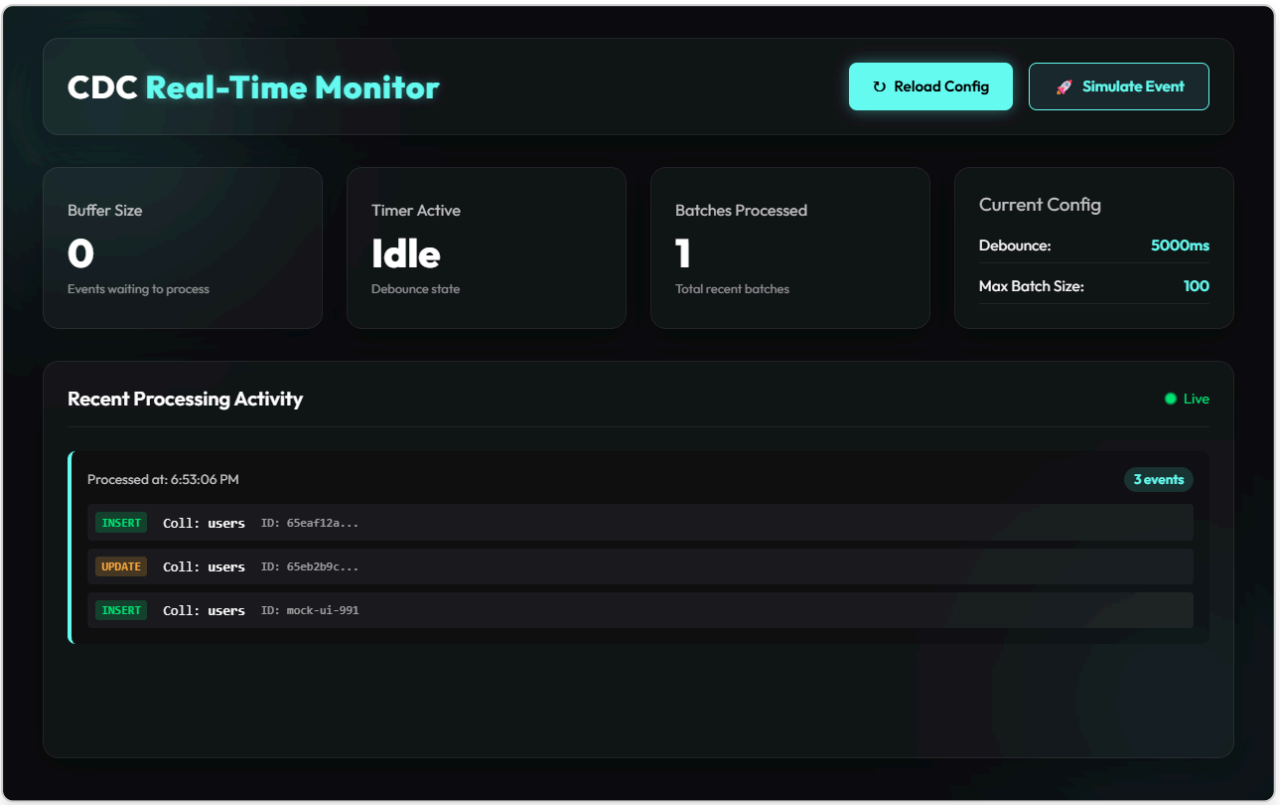

Figure 2: Real-time update displaying processed event batches

6. Conclusion and Future Scope

This project successfully implemented an optimized Change Data Capture stream. It proved that using MongoDB Change Streams with Node.js Event Buffering substantially reduces database fatigue, while delivering a beautifully responsive, zero-delay experience to the users.

Future Scope: The debounced batches can be easily integrated with a Pub/Sub queue system like Apache Kafka or Redis. This would enable broadcasting changes reliably to mobile applications, email servers, or multiple diverse microservices simultaneously.